CSC 242: Introduction to Computer Science II

Assignment 8

See D2L for due date

Reading

Review Chapter 10 and read Chapter 11 in **Introduction to Computing using Python: An Application Development Focus, Second Edition** by Ljubomir Perković.

Logistics

In this class programming assignments may be completed in consultation with up to two other classmates. You must identify the classmates with whom you collaborate in a comment at the top of the assignment, and the number of collaborators on any assignment **may not exceed two other people**. You must also submit a comment in your submission for each assignment that describes in detail how each collaborator contributed to the assignment. If you did not collaborate with anyone on the assignment, you must include a comment that says that. You may not under any circumstances discuss the assignments with classmates other than your identified collaborators. Working so closely with anyone other than your identified collaborators, Mr. Zoko, or the lab assistant, so as to produce identical or near identical code is a violation of the Academic Integrity policy. This policy will be strictly enforced.

Please include the following with your assignment submission:

1. A comment at the top of your Python file identifying any classmates with whom you discussed or in any other way collaborated on the assignment. You may work (directly or indirectly) with **no more than two** other people.
2. Add a comment at the top of your Python file that describes for each person what they contributed to the assignment. This must be at least 2-3 sentences and be **very specific and detailed**.

A submission that does not include a list of collaborators and a comment indicating how you collaborated with classmates will earn a 0. If you worked alone you must put a comment at the top of your file that indicates that or you will also receive a 0. There will be no exceptions to this rule.

Again, you are subject to all the rules specified in the Academic Integrity policy. Please read it carefully before beginning this assignment.

Assignment

Please remember that you are not allowed to consult online resources when completing homework assignments. If you have questions about this assignment, please contact me.

Implement the functions below in a file called **csc242hw8.py** a template for which has been provided on the D2L site.

The functions written for this assignment must be **recursive** and **must not** use global variables. Do not modify the function names or parameters in the template file. The recursive functions **may not use loops except to list the contents of a directory**. In some cases certain built-in Python functions are disallowed. None of the functions should alter the lists passed as parameters nor should the functions make copies of the lists in order to alter them. Only non-destructive solutions should be submitted. Please read the problem description carefully. Function that are described as returning values should not do any printing unless specified. Solutions that do not follow these guidelines will not earn full credit, even if they produce the correct results in all cases.

Please note that local variables are fine in any of the functions.

1. (20 points) Write a **recursive** function **printer**() that takes as parameters a pathname of a folder as a string and an integer indent and **prints** to the screen the pathname of every subfolder contained in the folder, directly or indirectly. The subfolder path names should be output with an indentation that is proportional to their depth with respect to the topmost folder and should use the parameter indent to achieve this. The example below illustrates the execution of the function on several sample folders. Two sets of folders (can be found in the zip file containing the template and posted to D2L:

   Reminder : See the antivirusredux.py example.

```
printer('count',5)
       count
         count/A
           count/A/A
             count/A/A/A
         count/C
           count/C/A
         count/B
           count/B/A
             count/B/A/A
           count/B/B
             count/B/B/A
               count/B/B/A/A
printer('Test',10)
            Test
              Test/Test1
                Test/Test1/Test1-A
                  Test/Test1/Test1-A/Test1-A-A
                    Test/Test1/Test1-A/Test1-A-A/A
                Test/Test1/Test1-C
                  Test/Test1/Test1-C/C1
                  Test/Test1/Test1-C/C3
                  Test/Test1/Test1-C/C2
                    Test/Test1/Test1-C/C2/C31
                      Test/Test1/Test1-C/C2/C31/A
                        Test/Test1/Test1-C/C2/C31/A/A
                          Test/Test1/Test1-C/C2/C31/A/A/A
                            Test/Test1/Test1-C/C2/C31/A/A/A/A
                    Test/Test1/Test1-C/C2/C32
                Test/Test1/Test1-B
                  Test/Test1/Test1-B/Test1-B-B
                  Test/Test1/Test1-B/Test1-B-A
                    Test/Test1/Test1-B/Test1-B-A/B3
                      Test/Test1/Test1-B/Test1-B-A/B3/B4
              Test/Test3
              Test/Test2
                Test/Test2/A
                  Test/Test2/A/B-1
                  Test/Test2/A/B-2
                Test/Test2/C
                Test/Test2/B
                  Test/Test2/B/B1-1
                    Test/Test2/B/B1-1/B2-2
                      Test/Test2/B/B1-1/B2-2/B
                        Test/Test2/B/B1-1/B2-2/B/B
```

2. (20 points) Write a **recursive** function **wordWithLetterCount** () that takes an arbitrarily nested list as a parameter, and a parameter for a letter to search for in strings. It **returns** the total count of the strings in the list that have the letter in it. You are not allowed to use any list methods in your function other than indexing (e.g. lst[i] for some integer i), slicing (e.g. lst[i:j] for some integers i and j). The function may not use a loop. Solutions that use built-in functions rather than recursion to solve the problem will earn very little credit. You may assume that the initial value given to the function is a list, and your function does not have to behave in any reasonable way when given something other than a list as a parameter. The list may contain values of any type and you must only take the number of the values that are strings (str). The type() function is helpful in determining what values can be found in the list. Strings found within other collections (tuples, dictionaries, etc.) should not be included in the sum. The following shows the behavior of the function on some sample parameters. Please remember that these examples are just that. Your function must work correctly on all valid parameters, and you may not make any assumptions about the number of nested sublists or the depth of the nesting in the list provided as a parameter to the function:

```
print(wordsWithLetterCount([['','cat',12.0,'dog','MOUSE',1,2,'bird',8,9,10]],'o'))

2
print(wordsWithLetterCount([],'o'))
0
print(wordsWithLetterCount([[[['COMPUTER','science',[[4]],[['animation','art']]]]]],'e'))

2
```

Problem 3 (20 points):

Implement a **recursive** function **uniqueWordsWithLetter**() that takes as parameters the name of a folder and a letter to search for in words in the file. The function returns a set of all the unique words found in the files in the folder and sub folders. Please note that your function must work as described on any directory structure, not just the one provided as an example. You must use recursion to go through the subfolders. You may not use python crawlers or apis (walk) other than the ones used in the antivirus-redux example to find files in sub directories. Doing so will be an automatic 0 for this problem. You may use a for loop to loop through the contents of a folder or file. The content is case sensitive, and you are not required to remove punctuation. The following illustrates several searches using a sample set of folders and directories located in the zip file containing the exam template. You ***don't*** need to worry about punctuation, case or file error handling for this problem.

Hints: You can create an empty set as follows: s=set()

You can combine two sets with the | operator (the key above the enter key. Example:

```
s1={1,2,3}
s2={2,3,4}
s=s1|s2
s
{1, 2, 3, 4}
```

```
uniqueWordsWithLetter('Test','u')
{'recursive', 'up'}
uniqueWordsWithLetter('Test','a')
{'and', 'call.', 'a', 'away', 'calls.', 'another'}
```

Problem 4 (20 points):

(20 Points): Write a **recursive** function **sizer** () that takes as a parameter a pathname of a folder as a string. You must return an int with the count of characters in all files you find within the folder and its subfolders. You must use recursion to go through the subfolders. You may not use python crawlers or apis (walk) other than the ones used in the antivirus-redux example to find files in sub directories. Doing so will be an automatic 0 for this problem. You may use a for loop to loop through the contents of a folder. The following illustrates several searches using a sample set of folders and directories located in the zip file containing the exam template. You ***don't*** need to worry about punctuation, case or file error handling for this problem.

```
print(sizer('Test'))
331
print(sizer('count'))
0
print(sizer('Test-2'))
500
```

Problem 5 (20 points):

Part 1 (20 Points): Write a **recursive** function **summary** () that takes as a parameter a pathname of a folder as a string. You must return a string with the contents of all files you find within the folder and its subfolders. You must use recursion to go through the subfolders. You may not use python crawlers or apis (walk) other than the ones used in the antivirus-redux example to find files in sub directories. Doing so will be an automatic 0 for this problem. You may use a for loop to loop through the contents of a folder. The following illustrates several searches using a sample set of folders and directories located in the zip file containing the exam template. You ***don't*** need to worry about punctuation, case or file error handling for this problem.

```
print(summary('Test'))
Yet another test of the recursive calls.Yet another test of the recursive calls.Yet another test of the recursive c
alls.Yet another test of the recursive calls.Yet another test of the recursive calls.3
Yet another test of the recursive calls.This is a test of the recursive call.
This is a test of the recursive call.up up and away
print(summary('count'))

print(summary('Test-2'))
I hope you like this assignment.Looking forward to spring break!This is a test. Zoko assignment.Looking forward to
spring break!Looking forward to spring break!This is a test. Zoko assignment.I hope you like this assignment.Lookin
g forward to winter break!I hope you like this assignment.This is a test. Zoko assignment.Last Day of Spring:3/22/2
019
April Fools:4/1/2019
Fourth of July:7/4/2019
I hope you like this assignment.Last Day of Spring:3/22/2026
April Fools:4/1/2026
Fourth of July:7/4/2026
```

Submitting the assignment

You must submit the assignment using the assignment 8 dropbox on the D2L site. Submit a Python file (csc242hw8.py) with your implementation in it and comments describing your collaboration status. Submissions after the deadline listed above will be automatically rejected by the system. See the syllabus for the grading policy.

Grading

The assignment is worth 100 points. Any student who does not submit comments in the Python file describing the contributions of each team member or indicating that he/she worked alone will earn a 0 on the assignment.